Aarhus University

Computer Science

Numerical Linear Algebra

Handin 7

Author
Søren M. Damsgaard

Student number
202309814

April 7, 2026

Goofy ah title of finding solutions and evaluating their trust or something

The purpose of this clown show is to revisit heatmaps and how to ascertain the accuracy of a QR and SVD decomposition solution via their condition number but in Python!

(a) - funny ah 'straight' equation

The illumination level of a square y_j is a product of $\frac{x_i}{d_{ij}^2} = y_j$ for every lamp x_i, and to show a linear relation for them the following linear equation will be used

$$Ax = y \tag{1}$$

where Ax must result in $\frac{x_i}{d_{ij}^2}$ which means equation (1) will have the following values.

$$\begin{pmatrix} \frac{1}{d_{0,0}^2} & \cdots & \frac{1}{d_{0,11}^2} \\ \vdots & \ddots & \vdots \\ \frac{1}{d_{599,0}^2} & \cdots & \frac{1}{d_{599,11}^2} \end{pmatrix}_{600\times 12} \begin{pmatrix} x_0 \\ \vdots \\ x_{11} \end{pmatrix}_{12\times 1} = \begin{pmatrix} y_0 \\ \vdots \\ y_{599} \end{pmatrix}_{600\times 1} \tag{2}$$

hence the Author states they are related linearly!
In Python the coefficient matrix A can be defined by defining the center of each lamp and using that to define $\frac{1}{d_{ij}^2}$ for each square represented by the matrix A.
Note that some lines of code might be removed in these small snippets of code; it's usually because the lines are not relevant to the question.

```
lamps = np.array([[2,3,3],[11,5,3.5],[15,2,4.5],[20,4,3.1],[4,13,3.7],[4,19,3],[12,12,4],[13,18,3.6


# Center in each square j
xCenter = np.arange(0.5,30,1)
yCenter = np.arange(0.5,20,1)
centers = np.array([[x,y,0] for x in xCenter for y in yCenter] ) # Takes the center for each 1 x 1 square in a 30(x) x 20(y) grid

# ---- Create the A matrix ----
A = np.zeros((600,12))
for i in range(600):
    for j in range(12):
        d = np.sum((centers[i] - lamps[j])**2)
        A[i,j] = 1 / d
```

See Appendix for complete code

(b) - plotting a (heat)et plan

The equation can't be plotted directly to a heatmap since the dimensions of y_j are 600×1, so it has to be reshaped to look like the parking lot, which is done with the following Python code.

```
strenght = np.zeros((12,1))
for i in range(12):
    strenght[i] = 20

y = A @ strenght

parkinglot = y.reshape(30,20)
pain = parkinglot.T
```

The plotting of the heatmap can be found in the Appendix, since it's really not that interesting.

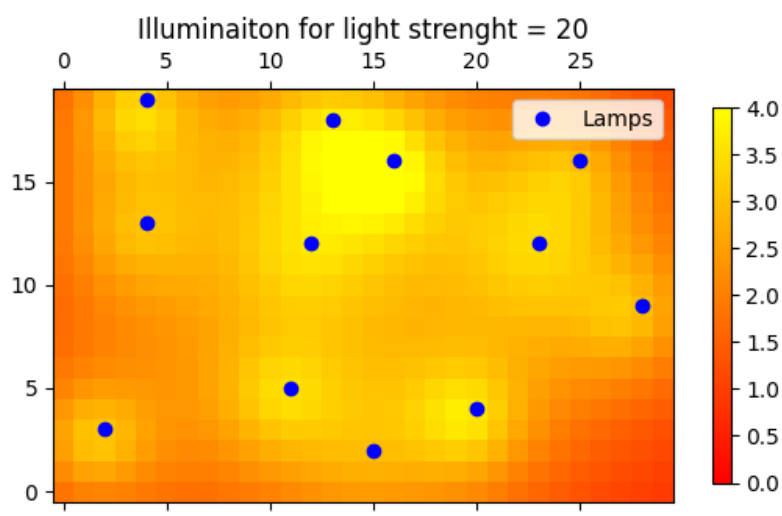

Figure 1: Heatmap of a parking lot with 12 lamps, of varying heights, each emitting a light of strength 20

(c-d) - QR and SVD on the case for the 1

To find a solution where each y_j is approximately 1, an appropriate vector x has to be found. There are two methods that can do this: SVD-decomposition and QR-decomposition.
Note that in both cases, when x has been found, the vector y is transformed into a matrix of the form 20×30 for the plotting.

QR-Decomposition

NumPy supplies the function NP.LINALG.LSTSQ which goes through the math and returns the solution for x in $Ax = y$.
Although this was not the intended way to do it, like it was described in 17.3 [1] for NP.LINALG.QR, reading the documentation of that led to the one used in this assignment.

```
targetY = np.ones((600,1))
QRresult = np.linalg.lstsq(A,targetY)
QRstrength = QRresult[0] # extract the actual strenght values, since lstsq
    outputs a variety of things
QRy = A @ QRstrength
QRparkinglot = QRy.reshape(30,20)
QRpain =QRparkinglot.T
```

In the heat plot for this and the next figure, the CMAP is being limited to the maximum and minimum values of illumination level y_j; these values are shown in section (d).

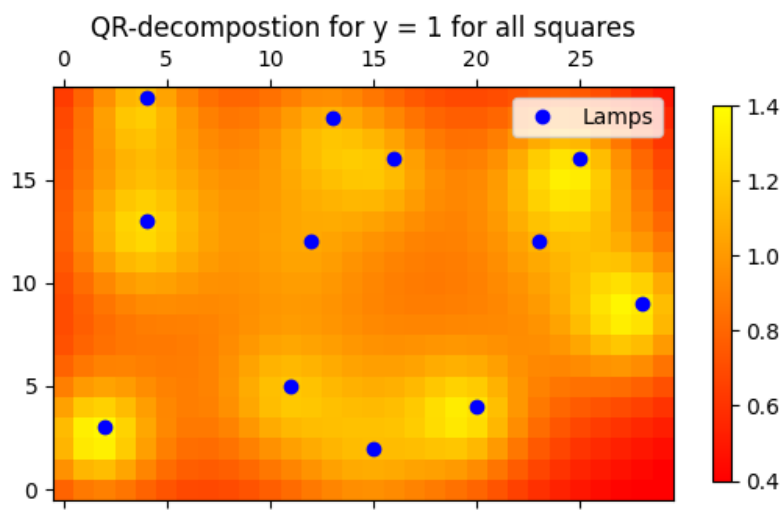

Figure 2: Approximation for $y_j = 1$ via QR-decomposition with a visual margin corresponding to the max and min of the values.

SVD-decomposition

For the SVD, the process from 16.3 [1] was followed.

```
U, S, VT = np.linalg.svd(A, full_matrices=False) # got dimension error so
    we limit the matrix
proj = U.T @ targetY
invS = np.diag(1/S)
b = invS @ proj
SVDstrenght = VT.T @ b
SVDy = A @ SVDstrenght
SVDparkinglot = SVDy.reshape(30,20)
SVDpain = SVDparkinglot.T
```

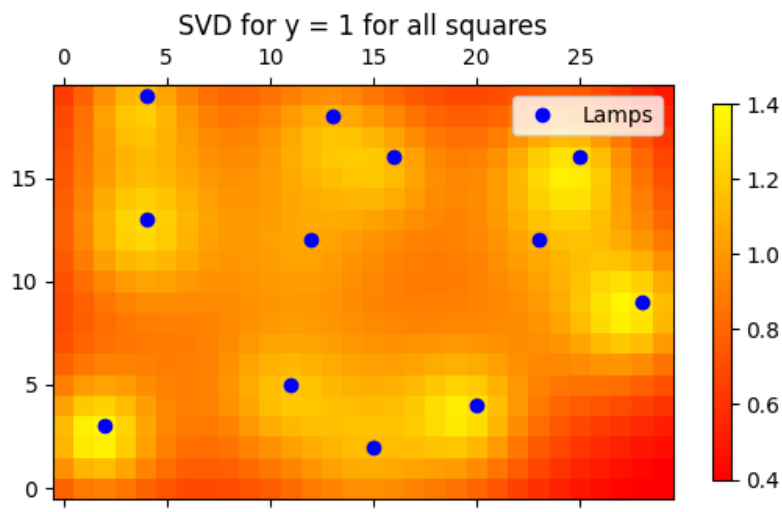

Figure 3: Approximation for $y_j = 1$ via QR-decomposition with a visual margin corresponding to the max and min of the values.

Realism and comparison of QR and SVD

Even though QR and SVD are good at finding solutions, it's not guaranteed the solution is 'realistic'; for example, a negative x_i would mean a lamp is somehow emitting darkness (edgy ah emo lamp stop that), so a sanity check is needed by checking $x_i > 0$.

```
print(f'QR realistic? {np.all(QRstrength > 0)}')
print(f'SVD realistic? {np.all(SVDstrenght > 0)}')
```

So both QR and SVD are realistic, but are they the same in regards to output?
If one looks back at the heat plots 2 and 3, one can visually ascertain that the difference in results is small, based on the limits set for the plots.
These limits approximately represent the smallest and largest value down to one decimal $\{y \in [0.4, 1.4]\}$, the more specific values are:

```
print(SVDpain.min(), SVDpain.max())
print(QRpain.min(), QRpain.max())
# Output is artificially inserted below for the assignment
QRmin = 0.3587581821413017   QRmax = 1.366147099159025
SVDmin = 0.3587581821413019  SVDmax = 1.3661470991590228
```

From this their likeness is shown and the maximal deviation is $1 - 0.3587 = 0.6413$, which tells us that the lamps can't provide an illumination of 1 evenly across the parking lot.

(e) - Interrogation for the instability of the results

Lastly, by looking at the condition number, it's possible to evaluate how accurate the computations of x are. The condition number κ is a multiplier of how big of an error can happen when computing x. Since y is constant and A varies, it can be hard to find exact values for κ, so an upper bound is needed; the formula from equation 17.5 [1] is used.

A lot of the code is taken directly from the examples given in chapter 17 [1].

```
kappa_a = S[0] / S[-1]
proj_y = U @ (U.T @ targetY)
cos_theta = np.linalg.norm(proj_y) / np.linalg.norm(targetY)
x = VT.T @ (np.diag(1/S) @ (U.T @ targetY))
eta = S[0] * np.linalg.norm(x) / np.linalg.norm(proj_y)
```

and the results are:

$$\kappa(A) = 8.389261e + 00 \qquad \cos(\theta) = 9.841725e - 01\ (10.21^\circ) \qquad \eta = 1.153937e + 00$$

For finding the upper bound for κ:

```
kond_x_a_max = (kappa_a
    + (kappa_a**2 * np.sqrt(1-cos_theta**2)
    / (eta * cos_theta)))

machine_epsilon = np.finfo(float).eps
estimatedPrecision = kond_x_a_max * machine_epsilon
```

With these results, we find an upper bound of $kond_x_a_max \approx 19.37$. Multiplying this by the machine epsilon gives an $estimatedPrecision \approx 4.30 \times 10^{-15}$, suggesting that our results are accurate to about 15 decimal points.
This also gives an answer as to how a change in A affects x since $kond_x_a_max \approx 19.37$ is a multiplier of how a change in A affects x. This means if A changed by 1%, then x would correspondingly change by a factor 19.37, resulting in a 19.37% change.

So in conclusion the approximation of QR and SVD in this case is quite good, based on their almost identical results that falls with the $estimatedPrecision$. Even thouhg there is a deviation, it is based on the physical limitation of the lamps placements, so all in all the the solution found is a good fit.

References

[1] Andrew Swan. *Numerisk Linær Algebra*. Institut for Matematik, Aarhus Universitet, 2025.

Appendix

```python
import numpy as np
import matplotlib.pyplot as plt

# Estimated coordinates for each lamp
lamps = np.array([[2,3,3],[11,5,3.5],[15,2,4.5],[20,4,3.1],[4,13,3.7],[4,19,3],[12,12,4],[13,18,3.6

lampsX = lamps[:,0]
lampsY = lamps[:,1]
# Center in each square j
xCenter = np.arange(0.5,30,1)
yCenter = np.arange(0.5,20,1)
centers = np.array([[x,y,0] for x in xCenter for y in yCenter] ) # Takes the center for each 1 x 1 square in a 30(x) x 20(y) grid

# ---- Create the A matrix ----
A = np.zeros((600,12))
for i in range(600):
    for j in range(12):
        d = np.sum((centers[i] - lamps[j])**2)
        A[i,j] = 1 / d

# Find better way?
strenght = np.zeros((12,1))
for i in range(12):
    strenght[i] = 20

y = A @ strenght

parkinglot = y.reshape(30,20)
pain = parkinglot.T

# ---- QR decomposition with least-squared method ----

# see 16.2 in notes
targetY = np.ones((600,1))
QRresult = np.linalg.lstsq(A,targetY)
QRstrength = QRresult[0] # extract the actual strenght values, since lstsq outputs a variety of things
QRy = A @ QRstrength
QRparkinglot = QRy.reshape(30,20)
QRpain =QRparkinglot.T

# ---- SVD type shiz ----
# using the same targetY as in the QR section
```

```python

# see 16.3 in notes
U, S, VT = np.linalg.svd(A, full_matrices=False) # got dimension error so we limit the matrix
proj = U.T @ targetY
invS = np.diag(1/S)
b = invS @ proj
SVDstrenght = VT.T @ b
SVDy = A @ SVDstrenght
SVDparkinglot = SVDy.reshape(30,20)
SVDpain = SVDparkinglot.T


# ---- check realism ----

print(f'QR realistic? {np.all(QRstrength > 0)}')
print(f'SVD realistic? {np.all(SVDstrenght > 0)}')

# min max value of 'y' for each method

print(SVDpain.min(), SVDpain.max())
print(QRpain.min(), QRpain.max())


# ---- Condition number ----
# Reusing the SVD from earlier and a lot of this is directly from the example in chapter 17

kappa_a = S[0] / S[-1] # how much A cause error??
proj_y = U @ (U.T @ targetY)
cos_theta = np.linalg.norm(proj_y) / np.linalg.norm(targetY)
x = VT.T @ (np.diag(1/S) @ (U.T @ targetY))
eta = S[0] * np.linalg.norm(x) / np.linalg.norm(proj_y)

print(f'{kappa_a = :e}')
print(f'{cos_theta = :e}')
print(f'Theta angle = {np.degrees(np.arccos(cos_theta)):.2f} degrees')
print(f'{eta = :e}')


# how many digits i can trust?
kond_x_y = kappa_a / (eta * cos_theta)

kond_x_a_max = (kappa_a
+ (kappa_a**2 * np.sqrt(1-cos_theta**2)
/ (eta * cos_theta)))

print(f'{kond_x_y = :e}')
print(f'{kond_x_a_max = :e}')

machine_epsilon = np.finfo(float).eps
estimatedPrecision = kond_x_a_max * machine_epsilon

print(f'{estimatedPrecision = :e}')
```

```python


# ---- Plotting ----

# Heatmap for (b)
fig1, heat = plt.subplots()
im1 = heat.matshow(pain, cmap='autumn', clim=(0,4), origin='lower') # I hate this...y-axis won't move down ;_;
fig1.colorbar(im1, shrink=0.65)
heat.plot(lampsX,lampsY, 'o', color='blue', label='Lamps')
heat.set_title('Illumination for light strength = 20')
heat.legend()

# QR
fig2, QRHeat = plt.subplots()
im2 = QRHeat.matshow(QRpain ,cmap='autumn', clim=(0.4,1.4), origin='lower', ) # I hate this...y-axis won't move down ;_
fig2.colorbar(im2, shrink=0.65)
QRHeat.plot(lampsX,lampsY, 'o', color='blue', label='Lamps')
QRHeat.set_title('Averaging illumination strength to 1 via QR-decomp')
QRHeat.legend()

# SVD
fig3, SVDHeat = plt.subplots()
im3 = SVDHeat.matshow(SVDpain ,cmap='autumn', clim=(0.4,1.4), origin='lower') # I hate this...y-axis won't move down ;_;
fig3.colorbar(im3, shrink=0.65)
SVDHeat.plot(lampsX,lampsY, 'o', color='blue', label='Lamps')
SVDHeat.set_title('Averaging illumination strength to 1 via SVD-decomp')
SVDHeat.legend()

plt.show()
```